MedVault Security System Guide

Technical Lexicon, Code Implementations & Architecture Reference

Author: MedVault Development Team | **Standards:** ISO/IEC 25010:2023, OWASP Top 10, NIST CSF, CIS Ubuntu |
Target Audience: Defense Panel, Evaluators, Engineers

This document provides a comprehensive, rigorous dictionary and explanation of the **phrases, programmatic codes, cryptographic algorithms, networking directives, and architectural terminologies** implemented across the MedVault healthcare database and API service. Each entry explains the core definition, its purpose in healthcare information assurance, and how it is concretely realized in MedVault's source files.

1. Cryptographic Pipelines & Data-at-Rest Protection

AES-256-GCM (Advanced Encryption Standard in Galois/Counter Mode)

Definition: An authenticated symmetric cipher utilizing a 256-bit key. Unlike basic encryption modes (like CBC), GCM simultaneously guarantees both *confidentiality* (preventing unauthorized viewing) and *integrity/authenticity* (detecting any modification of data).
Application in MedVault: Used in `frontend/server/src/security.js` to encrypt sensitive Electronic Health Records (EHR) before they touch SQLite. If an unauthorized actor reads or modifies the database file, decryption fails automatically.

```
const cipher = crypto.createCipheriv('aes-256-gcm', encryptionKey, iv);
cipher.setAAD(Buffer.from(context));
```

IV (Initialization Vector) / Nonce (96-bit)

Definition: A cryptographically secure random number generated once for every single encryption operation. Its purpose is to ensure that encrypting the exact same patient value twice produces completely different ciphertext outputs, defeating pattern recognition attacks.
Application in MedVault: Generated per record using `crypto.randomBytes(12)` (96 bits) and stored as Base64 beside the ciphertext. Never reused with the same key.

```
const iv = crypto.randomBytes(12); // Unique 96-bit IV per encrypted field
```

Authentication Tag (128-bit)

Definition: A cryptographic signature generated during GCM mode encryption. Upon decryption, the cipher recalculates this tag; if even a single bit of the stored ciphertext was altered (bit-flipping attack), the decipher rejects the record and throws an authentication error.
Application in MedVault: Generated with `cipher.getAuthTag()` and validated with `decipher.setAuthTag()` in `security.js`.

```
decipher.setAuthTag(Buffer.from(value.tag, 'base64')); // Prevents ciphertext tampering
```

AAD (Additional Authenticated Data) & Context Binding

Definition: Data that is not encrypted itself, but is bound cryptographically to the ciphertext. This ensures that encrypted data cannot be extracted from one patient record and pasted into another patient's field (ciphertext transplantation attack).
Application in MedVault: Context is bound as `medvault:{record_id}.{field_name}`. If ciphertext from Patient A is moved to Patient B, decryption will abort.

```
cipher.setAAD(Buffer.from(`medvault:${object?.id || ''}.${field}`));
```

Field-Level Encryption (FLE)

Definition: A targeted security architecture where sensitive attributes (diagnoses, medical history, vitals, contact details) are encrypted individually, while non-sensitive structural keys (record IDs, status, timestamps) remain queryable.
Application in MedVault: Managed via a protected fields whitelist (`vitals`, `diagnosis`, `treatment`, `notes`, etc.). Ensures fast indexing while preserving HIPAA/data privacy compliance.

```
const protectedFields = new Set(['vitals', 'medical', 'diagnosis', 'treatment', 'notes', ...]);
```

Bcrypt Password Hashing & Random Salting (Cost Factor 12)

Definition: An adaptive, one-way cryptographic hash function designed specifically for passwords. A 'salt' is a 128-bit random value prepended to the password before hashing to defeat precomputed Rainbow Table attacks. Cost factor 12 forces $2^{12} = 4,096$ hashing rounds to resist GPU brute-force attacks.
Application in MedVault: Implemented in `hashPassword()` using `bcrypt.hash(password, 12)` for all staff credentials.

```
const hashPassword = password => bcrypt.hash(password, 12);
```

SHA-256 Digest & Hash Chaining (Tamper-Evident Ledger)

Definition: An append-only data structure where every new audit event includes the SHA-256 hash of the immediately preceding event. If an attacker tampers with or deletes a historic row in the database, the hash chain breaks from that point forward.
Application in MedVault: Defined in `auditDigest()` and enforced by SQLite database triggers (`audit_no_update`, `audit_no_delete`) that abort modification attempts.

```
entry.prev_hash = previous?.entry_hash || '';
entry.entry_hash = auditDigest(entry);
```

2. Authentication, Session Management & Access Control

JWT (JSON Web Token) with HS256 Signing

Definition: A compact, URL-safe means of representing claims between two parties. The payload is signed with HMAC-SHA256 using a private 256-bit server key. In MedVault, the token contains standard claims: `sub` (username), `role`, `jti` (unique token UUID), `iss` ('medvault'), `aud` ('medvault-web'), and `exp` (expiry timestamp).
Application in MedVault: Handled in `security.js` via `jwt.sign()` and `jwt.verify()`.

```
jwt.sign({ role }, jwtKey, { algorithm: 'HS256', subject: username, jwtid: crypto.randomUUID(), ... })
```

Cookie Security Flags: HttpOnly, SameSite=Strict, Secure

Definition: Security attributes enforced on HTTP cookies:
• **HttpOnly:** Forbids client-side JavaScript (`document.cookie`) from reading the cookie, completely mitigating session theft via XSS.
• **SameSite=Strict:** Forbids the browser from sending the cookie in any cross-site requests, neutralizing CSRF attacks.
• **Secure:** Ensures the browser will only transmit the cookie over encrypted HTTPS connections.
Application in MedVault: Configured on `medvault_session` and `medvault_csrf` cookies.

```
Set-Cookie: medvault_session=...; HttpOnly; SameSite=Strict; Path=/; Secure; Max-Age=28800
```

Idle Timeout (30 min) vs. Absolute Session Expiration (8 hours)

Definition: Defense against session hijacking on unattended clinic workstations. The absolute limit terminates a session after 8 hours regardless of activity. The idle timeout terminates a session if no trusted user interactions occur for 30 consecutive minutes.
Application in MedVault: The server inspects `last_seen_at` in the `sessions` table. Automated background UI polls send `x-medvault-background: 1`, which intentionally does *not* reset the idle timer.

```
if (request.get('x-medvault-background') !== '1')
await db.prepare('UPDATE sessions SET last_seen_at = ? WHERE token_hash = ?').run(Date.now(),
tokenHash);
```

Brute-Force Account Lockout (5-Failure Policy)

Definition: An access control mechanism that limits consecutive credential guessing. After 5 incorrect password attempts on a normalized username, further attempts are blocked for 15 minutes, returning HTTP 429.
Application in MedVault: Tracked in the `login_attempts` table across IP address changes. To prevent timing attacks (user enumeration), unknown usernames execute a dummy bcrypt hash computation to preserve uniform response time.

```
if (attempt.count >= 5) return response.status(429).setHeader('Retry-After', '900').json({ error:
'Account locked.' });
```

RBAC (Role-Based Access Control) & Principle of Least Privilege

Definition: Access control model where permissions are attached to user roles rather than individual users. Users are granted only the minimum access level necessary to complete their assigned functions.
Application in MedVault: Governed by `authenticate` and route middlewares in `index.js`. Super Admins manage staff and doctors but cannot read clinical EHR; Receptionists manage bookings and demographics; Medical Secretaries manage clinical histories and assigned doctor schedules.

```
if (request.user.role !== 'Super Admin') return reject(request, response, 403, 'Super Admin access
required.');
```

BOLA (Broken Object Level Authorization) / IDOR Mitigation

Definition: Vulnerability where an attacker changes an ID parameter in an API request to view or edit another user's or doctor's data without authorization.
Application in MedVault: Route handlers verify object ownership. Medical secretaries can only modify schedules of physicians assigned directly to their username.

```
if (doctor.secretary_username !== request.user.username) return reject(request, response, 403, 'Doctor
not assigned.');
```

3. Application-Level Defenses & Web Security

Parameterized Queries / Prepared Statements (SQLi Prevention)

Definition: A method of executing database statements where SQL code structure is compiled separately from user input. Inputs are treated strictly as inert literals rather than executable commands, rendering SQL Injection impossible.
Application in MedVault: Enforced across all database interactions in `storage.js` and `index.js`. No dynamic SQL string concatenation exists.

```
db.prepare('SELECT role, status FROM staff_accounts WHERE username = ?').get(username)
```

Cross-Site Scripting (XSS) Defenses (Sanitization + Contextual Escaping)

Definition: Attack where malicious scripts are injected into web pages viewed by other users. Defended in MedVault via a two-layer defense:
1. **Input Sanitization:** Inbound text is stripped of all HTML tags, attributes, and disallowed ASCII/Unicode control characters via `sanitize-html` and `entities.decodeHTML`.
2. **Output Escaping:** The React client automatically treats all rendered variables as text literals rather than HTML, preventing browser script execution.

```
sanitize(plain, { allowedTags: [], allowedAttributes: {} })
```

CSP (Content Security Policy) Headers

Definition: An HTTP response header allowing site administrators to declare approved sources of content that the browser is permitted to load, blocking unauthorized inline scripts and external malicious assets.
Application in MedVault: Configured using `helmet()` in `security.js`.

```
contentSecurityPolicy: { directives: { defaultSrc: ["'self'"], scriptSrc: ["'self'"], frameAncestors: ["'none'"] } }
```

Anti-CSRF (Cross-Site Request Forgery) Double-Submit Token Pattern

Definition: Prevents unauthorized web apps from executing state-changing mutations on behalf of an authenticated user. MedVault generates a cryptographically random 64-hex token stored in a Strict cookie, which must match the custom header `X-CSRF-Token` on every mutation (POST/PATCH/DELETE).
Application in MedVault: Checked in `installProtection()`. Cross-site requests (`sec-fetch-site: cross-site`) and foreign origins are denied immediately.

```
if (!cookie || !/^[a-f0-9]{64}$/.test(token) || cookie !== token) return response.status(403);
```

Optimistic Concurrency Control (Conflict Checking)

Definition: A concurrency control pattern ensuring that simultaneous edits by multiple staff members do not overwrite each other silently. Requests supply a `before` snapshot of the record, which must match current storage.
Application in MedVault: Applied to appointments and patient records; mismatched versions are rejected with HTTP 409 Conflict.

```
if (!isDeepStrictEqual(currentRecord, request.body.before)) return response.status(409).json({ error: 'Record was modified by another user.' });
```

4. Infrastructure Hardening, Linux OS & Reverse Proxy

NGINX Reverse Proxy & TLS 1.3 Termination

Definition: An intermediate proxy server receiving public internet traffic, terminating SSL/TLS encryption, and forwarding clean requests to the internal Node.js container over an isolated network.
Application in MedVault: Configured in `deploy/nginx.conf`. Enforces `ssl_protocols TLSv1.3;`, HTTP 308 redirection to HTTPS, and hides backend internal topology.

```
server { listen 8443 ssl; ssl_protocols TLSv1.3; proxy_pass http://api:3000; }
```

Rate Limiting & Connection Throttling (Layer 7 DoS Protection)

Definition: Throttling mechanisms in NGINX that meter the frequency of incoming HTTP requests per client IP address to prevent denial of service and server saturation.
Application in MedVault: Limits API routes to 10 req/sec (burst 20) and sign-in endpoints to 5 req/min (burst 5). Limits simultaneous TCP connections per IP to 30.

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
limit_conn connections 30;
```

UFW (Uncomplicated Firewall) Default DENY Policy

Definition: Host-level packet filter configuration on Ubuntu Linux that blocks all incoming unsolicited network packets by default, permitting traffic only on explicitly whitelisted ports.
Application in MedVault: Implemented in `deploy/scripts/harden-host.sh`, opening only Port 80 (HTTP redirect), Port 443 (HTTPS), and Port 2222 (hardened SSH).

```
ufw default deny incoming
ufw allow 80/tcp && ufw allow 443/tcp && ufw allow 2222/tcp
```

Fail2Ban Dynamic Intrusion Prevention

Definition: An intrusion prevention software framework that inspects system logs for repeated authentication failures and automatically installs temporary firewall block rules for offending IP addresses.
Application in MedVault: Monitors `medvault-api` logs in systemd-journal. Offending IPs exceeding 5 failures in 15 minutes are banned for 1 hour in Docker's `DOCKER-USER` iptables chain.

```
[medvault]
enabled = true
maxretry = 5
findtime = 15m
bantime = 1h
action = iptables-allports[name=medvault, chain=DOCKER-USER]
```

Docker Container Hardening (Non-Root, Read-Only, Cap Drop)

Definition: Defense-in-depth container configurations that prevent attackers from escaping a compromised container onto the host machine:
• **Non-Root Execution:** Containers run as unprivileged users (UID 1000 for Node, UID 101 for NGINX).
• **Read-Only Root Filesystem (`read_only: true`):** Prevents attackers from downloading or installing malware in the container.
• **Capability Dropping (`cap_drop: [ALL]`):** Strips all Linux root privileges (e.g., raw sockets, kernel module loading).
• **Resource Limits:** Strict CPU and RAM caps (`cpus: 1.0, mem_limit: 512m`) prevent resource exhaustion.

```
services:
api:
read_only: true
cap_drop: [ALL]
security_opt: [no-new-privileges:true]
cpus: 1.0
mem_limit: 512m
```

SSH Hardening (Port 2222, RSA 4096-bit, PermitRootLogin no)

Definition: Server management configuration preventing unauthorized remote console access. Eliminates dictionary password brute forcing by requiring 4096-bit cryptographic RSA keys and moving off standard port 22.
Application in MedVault: Enforced in `deploy/sshd.conf` and verified by `harden-host.sh`.

```
Port 2222
PermitRootLogin no
PasswordAuthentication no
RequiredRSASize 4096
AuthenticationMethods publickey
```

5. Database Architecture, Backup & Disaster Recovery

SQLite WAL (Write-Ahead Logging) Mode

Definition: A high-performance concurrency mode where read operations do not block write operations, and write operations do not block reads. Changes are written sequentially to a separate WAL log before being checkpointed to the main database file.
Application in MedVault: Initialized at startup via `PRAGMA journal_mode = WAL;` alongside `PRAGMA busy_timeout = 5000;`.

```
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;
PRAGMA secure_delete = ON;
```

PRAGMA secure_delete = ON

Definition: An SQLite database pragma that forces deleted data pages to be actively overwritten with zeros, ensuring discarded patient records cannot be recovered by scanning raw unallocated disk sectors.
Application in MedVault: Configured during storage initialization in `index.js`.

```
await db.exec('PRAGMA secure_delete = ON;');
```

AES-256-GCM Encrypted Daily Backup Snapshot (.mvb)

Definition: Scheduled backup process capturing consistent database state using SQLite's online backup API, verifying table and audit chain integrity, streaming the snapshot through AES-256-GCM encryption with a dedicated backup key, and producing a SHA-256 checksum.
Application in MedVault: Scheduled at 02:00 Asia/Manila via `deploy/medvault-backup.cron` and executed by `frontend/server/scripts/backup.cjs`.

```
0 2 * * * root /usr/bin/flock -n /run/backup.lock docker compose run --rm backup
```

Pre-Restore Snapshot & Rollback Drill

Definition: Failsafe restoration protocol. Before restoring a backup file, the system creates a consistent `.pre-restore-*` snapshot of the active database. If the restored file is corrupted or fails authentication, the active database is preserved unchanged.
Application in MedVault: Implemented in `backup.cjs restore` and verified by `src/backup.test.js`.

```
docker compose run --rm -e MEDVAULT_CONFIRM_RESTORE=yes backup restore SELECTED_BACKUP.mvb
```

RPO (Recovery Point Objective) & RTO (Recovery Time Objective)

Definition: Core business continuity metrics:
• **RPO (Recovery Point Objective):** Maximum acceptable period of data loss measured in time. MedVault target: ≤ 24 hours (governed by daily 02:00 backups).
• **RTO (Recovery Time Objective):** Maximum acceptable duration of system downtime to restore operations. MedVault target: ≤ 60 minutes.

```
Target RPO: ≤ 24 Hours | Target RTO: ≤ 60 Minutes
```

6. Quality Models, Security Frameworks & Metrics

ISO/IEC 25010:2023 Product Quality Model

Definition: International standard establishing software product quality characteristics:
• **Confidentiality:** Data accessible only to authorized entities (AES-256-GCM + RBAC).
• **Integrity:** Protection against unauthorized alteration (GCM auth tags, SHA-256 audit chaining).
• **Non-Repudiation:** Ability to prove actions occurred (Cryptographic audit logs with timestamps and actor attribution).
• **Accountability:** Actions traced uniquely to an entity (Session binding and immutable logs).
• **Authenticity:** Verification of entity identity (Bcrypt password salting and JWT signatures).
• **Resistance:** Ability to sustain operations under attack (Fail2Ban, rate limiting, and container isolation).
• **Performance Efficiency:** Measured via load testing (MedVault achieved p95 = 434.72 ms vs. < 500 ms target).

OWASP Top 10 (2021) & OWASP API Security Top 10

Definition: Industry-standard reference for critical web and API security risks. MedVault addresses:
• **A01: Broken Access Control:** Enforced via route-level RBAC and ownership verification.
• **A02: Cryptographic Failures:** Enforced via AES-256-GCM at rest and TLS 1.3 in transit.
• **A03: Injection:** Enforced via SQLite prepared statements.
• **A04: Insecure Design:** Enforced via fail-closed startup and account lockout policies.
• **A05: Security Misconfiguration:** Enforced via Helmet CSP, non-root containers, and UFW default deny.
• **A07: Identification & Auth Failures:** Enforced via bcrypt salting, JWT cookies, and 15-minute lockouts.
• **A09: Logging & Monitoring Failures:** Enforced via SHA-256 chained audit events in systemd-journal.

NIST Cybersecurity Framework (CSF)

Definition: National Institute of Standards and Technology guidelines organizing security into five continuous functions:
1. **Identify:** Asset inventory, EHR classification, CIA baseline matrix.
2. **Protect:** AES-256-GCM encryption, TLS 1.3, RBAC, UFW firewall, container drop capabilities.
3. **Detect:** Structured JSON logging to systemd, audit chain startup verification.
4. **Respond:** Dynamic Fail2Ban IP blocking, immediate session revocation on role changes.
5. **Recover:** Encrypted daily online database dumps and pre-restore rollback procedures.

CIA Triad (Confidentiality, Integrity, Availability)

Definition: The foundational model for information security:
• **Confidentiality:** Preserving authorized restrictions on access and disclosure.
• **Integrity:** Guarding against improper data modification or destruction.
• **Availability:** Ensuring timely and reliable access to and use of information.

7. Quick Reference: Security Controls Matrix

Category	Primary Mechanism	Standard / Metric	Source File / Config
Data at Rest	AES-256-GCM field encryption	NIST SP 800-38D	`security.js`
Data in Transit	HTTPS / TLS 1.3 only	RFC 8446	`nginx.conf`
Passwords	Bcrypt (cost 12) + salt	OWASP Password Storage	`security.js`
Sessions	JWT in HttpOnly/Strict cookies	30m idle / 8h max	`index.js`
Lockout	5 failures = 15m lockout	HTTP 429 Retry-After	`index.js`
Injection	Parameterized queries	Zero SQLi	`storage.js`
Audit Trail	SHA-256 hash chaining + triggers	Append-only	`audit.js / storage.js`
Host Defense	UFW default DENY; port 2222	CIS Ubuntu Benchmark	`harden-host.sh`
DDoS (Layer 7)	NGINX rate limit (10r/s) + Fail2Ban	1-hour firewall jail	`nginx.conf / fail2ban`
Disaster Recovery	Daily encrypted backup cron (02:00)	RPO ≤ 24h, RTO ≤ 60m	`backup.cjs / cron`
Performance	Concurrent load testing (20 clients)	p95 = 434.72 ms (<500ms)	`load.cjs`